

Referință vs Instanță vs Instanțiere

Cum stau obiectele în memorie — teorie + exerciții (model: clasa Masina)

MyCodeSchool

Trei cuvinte pe care le confunzi mereu

Când scrii un singur rând:

```
Masina a = new Masina();
```

se întâmplă **trei lucruri diferite**, cu trei nume diferite:

Termen	Ce este	În exemplul de sus
Instanțiere	<i>acțiunea</i> de a crea un obiect, cu new	new Masina()
Instanță	<i>obiectul concret</i> care apare în memorie	obiectul Masina creat pe heap
Referință	<i>variabila</i> care „arată către” obiect	a

Regula de aur:

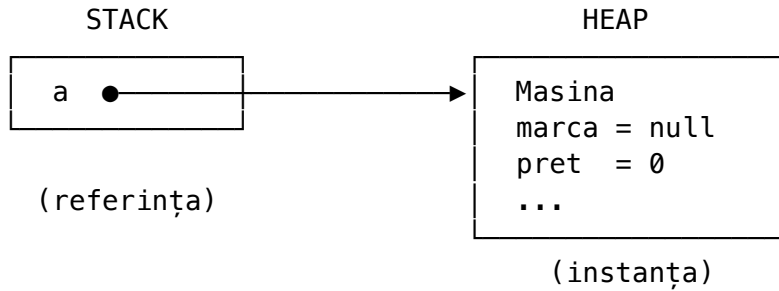
new creează **instanța** (obiectul din memorie). Variabila (a) este doar **referința** — o săgeată către obiect, **nu** obiectul în sine.

Unde stau, de fapt, în memorie

Java împarte memoria în două zone:

- **Stack** — aici stau variabilele locale (referințele). Mic, rapid.
- **Heap** — aici stau obiectele (instanțele) create cu new. Mare.

Pentru `Masina a = new Masina();`:



- a (referința) stă pe **stack** și conține **adresa** obiectului.
- Obiectul Masina (instanța) stă pe **heap**.
- a **nu conține** mașina — conține doar „unde s-o găsești”.

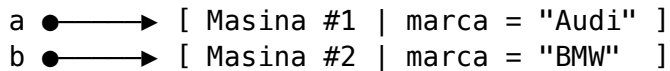
Analogie: instanța e **casa**, referința e **adresa scrisă pe un bilețel**. Poți avea mai multe bilețele cu aceeași adresă — dar casa rămâne una singură.

Momentul-cheie: `b = a` NU creează un obiect nou

Aici pică toată lumea. Comparăm două situații.

Situația A — două instanțe (două `new`)

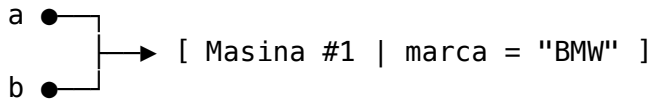
```
Masina a = new Masina();  
Masina b = new Masina(); // al doilea new -> a doua instanță  
a.marca = "Audi";  
b.marca = "BMW";
```



Două obiecte separate. Modific unul, celălalt nu se schimbă.

Situația B — o instanță, două referințe (`b = a`)

```
Masina a = new Masina();  
Masina b = a; // NICIUN new -> nu se creează obiect nou!  
a.marca = "Audi";  
b.marca = "BMW"; // modific prin b...  
System.out.println(a.marca); // ...dar se vede prin a -> afișează "BMW"
```



`a` și `b` sunt **două bilețele cu aceeași adresă**. Un singur obiect. Fenomenul se numește **aliasing**. Modifici obiectul prin oricare referință — schimbarea e vizibilă prin toate.

Ai văzut asta chiar în proiectul tău, în `ContBancar` (comentat în `Main.ex4`): `ContBancar cont2 = cont1; cont2.setTitular("ana");` — `cont1` și `cont2` sunt **același cont**, nu două conturi.

== compară referințe, nu conținut

```
Masina a = new Masina();  
Masina b = new Masina();  
Masina c = a;
```

```
a == b    // false -> adrese diferite (două instanțe)  
a == c    // true  -> aceeași adresă (aceeași instanță)
```

== întreabă „sunt același obiect?” (aceeași adresă), **nu** „au aceleași date?”. Chiar dacă a și b au exact aceleași atribute, a == b e false.

Ca să compari **conținutul**, ai nevoie de metoda equals() (pe care o scrii tu în clasă). De aceea la String se folosește .equals(), nu ==.

null — o referință fără obiect

```
Masina a = null;    // referință care NU arată către nimic  
a.getMarca();       // NullPointerException
```

null = bilețel gol, fără adresă. Dacă încerci să apelezi o metodă pe el, crapă. O instanță costă memorie; o referință null nu are obiect în spate.

Ce se întâmplă când dai obiectul unei metode

```
void ieftineste(Masina m) {  
    m.pret = 0;           // modifică obiectul -> se vede în afară  
    m = new Masina();     // reasignează referința locală -> NU se vede în afară  
}
```

Java trimite metodei o **copie a referinței** (nu a obiectului). Copia arată către **aceeași instanță**, deci poți modifica obiectul. Dar dacă îi pui copiei altă adresă (m = new ...), originalul din afară rămâne neatins.

Exerciții — prezici rezultatul

Nu rula codul. Scrie pe hârtie ce afișează și **de ce**. Soluțiile sunt la final.

E1. Câte instanțe?

```
Masina a = new Masina();
Masina b = new Masina();
Masina c = b;
```

Câte **instanțe** (obiecte pe heap) există? Câte **referințe**?

E2. Aliasing

```
Masina a = new Masina();
a.pret = 10000;
Masina b = a;
b.pret = 20000;
System.out.println(a.pret);
```

Ce se afișează?

E3. Instanțe separate

```
Masina a = new Masina();
a.marca = "Dacia";
Masina b = new Masina();
b.marca = "Dacia";
System.out.println(a == b);
System.out.println(a.marca == b.marca);
```

Ce afișează cele două rânduri?

E4. null

```
Masina a = new Masina();
Masina b = a;
a = null;
System.out.println(b.marca);
```

Compilează? Crapă? Ce afișează? (atenție: ce arată b?)

E5. Parametru de metodă

```
static void schimba(Masina m) {
    m.marca = "Ford";
    m = new Masina();
    m.marca = "Opel";
}
// în main:
Masina x = new Masina();
```

```
x.marca = "Audi";  
schimba(x);  
System.out.println(x.marca);
```

Ce afișează? De ce nu „Opel”?

E6. Lista din proiectul tău

În `MasinaService.afiseazaMasiniOrdonateDupaPret` ai:

```
Masina aux = masini.get(i);  
masini.set(i, masini.get(j));  
masini.set(j, aux);
```

`aux` este o **copie a mașinii** sau o **referință la aceeași mașină**? Ce s-ar întâmpla dacă, în loc de swap, ai scrie `masini.get(i).pret = 0`?

E7. Instanțiere fără referință

```
new Masina();
```

E cod valid? Se creează o instanță? O mai poți folosi după aceea?

Exerciții — scrie cod

E8. Demonstrează aliasing-ul

Scrie o metodă `demoAliasing()` care creează o `Masina`, o copiază într-o a doua referință fără `new`, modifică prețul prin a doua referință și **dovedește** cu un `println` că prima referință „vede” schimbarea.

E9. Demonstrează instanțe separate

Scrie `demoInstanteSeparate()`: două obiecte `Masina` cu `new`, aceleași valori în atribute, și arată că `a == b` e false.

E10. Numără instanțele

Adaugă în `Masina` un câmp static `int nrInstante`; și incrementează-l în constructor. Creează 3 mașini și afișează `Masina.nrInstante`. Explică de ce e static (aparține clasei, nu unei instanțe).

Soluții

E1. Două instanțe (două new) și **trei** referințe (a, b, c). c = b nu creează obiect — b și c arată către aceeași instanță.

E2. 20000. b = a -> o singură instanță, două referințe (aliasing). Modificarea prin b se vede prin a.

E3. false, apoi false. Sunt două instanțe diferite (== pe obiecte compară adresele). La marca, deși textul e „Dacia” în ambele, sunt String-uri obținute independent — nu te baza niciodată pe == la String, folosește .equals(). (Notă: cu literalii identici interniți ai putea primi true, exact „norocul” care ascunde bug-ul din MasinaService — motiv în plus să folosești .equals().)

E4. Compilează și **NU** crapă. Afișează null (valoarea câmpului marca, care n-a fost setat). a = null doar rupe săgeata lui a; b arată în continuare către obiect, care e viu. Ar crăpa doar dacă am face b = null; b.marca.

E5. Afișează Ford. Metoda primește o copie a referinței care arată către același obiect -> m.marca = "Ford" modifică obiectul original. Apoi m = new Masina() mută **doar copia locală** pe alt obiect; x din main rămâne pe primul. „Opel” se pune pe obiectul nou, aruncat la sfârșitul metodei.

E6. aux este o **referință** la aceeași mașină din listă (nu o copie a obiectului). Swap-ul e corect pentru că mutăm referințele în listă. Dar masini.get(i).pret = 0 ar modifica **obiectul real** din listă — pentru că get(i) întoarce referința, nu o copie.

E7. Valid. Se creează o instanță pe heap, dar **fără referință** către ea -> nu o mai poți accesa niciodată (devine „gunoi”, o va curăța Garbage Collector-ul). Are sens doar dacă constructorul face ceva cu efect (ex. un System.out.println).

E8 (model)

```
static void demoAliasing() {
    Masina a = new Masina();
    a.pret = 10000;
    Masina b = a;           // fără new -> aceeași instanță
    b.pret = 20000;
    System.out.println("a.pret = " + a.pret); // 20000 -> aliasing dovedit
}
```

E9 (model)

```
static void demoInstanteSeparate() {
    Masina a = new Masina();
    Masina b = new Masina();
    a.marca = "Audi";
    b.marca = "Audi";
    System.out.println(a == b); // false -> două instanțe
}
```

E10 (model)

```
public class Masina {
    static int nrInstante;           // aparține CLASEI, comun tuturor obiectelor
}
```



```
    public Masina() { nrInstante++; }  
}  
// ...  
new Masina(); new Masina(); new Masina();  
System.out.println(Masina.nrInstante);    // 3
```

static = un singur contor, ținut de clasă, nu câte unul per instanță. De aceea îl accesezi prin `Masina.nrInstante`, nu prin `a.nrInstante`.

De reținut (cheat sheet)

`new Masina()` → INSTANȚIERE (acțiunea)
obiectul de pe heap → INSTANȚĂ (rezultatul)
variabila `Masina a` → REFERINȚĂ (săgeata către obiect)

`Masina b = a;` → 0 instanțe noi, 2 referințe, 1 obiect (ALIASING)
`Masina b = new Masina();` → 1 instanță nouă

`==` → compară REFERINȚE (aceeași adresă?)
`equals` → compară CONȚINUT (aceleași date?)
`null` → referință fără obiect → `NullPointerException` dacă o folosești

- Un `new` = o instanță. Fără `new`, nu apare obiect nou.
- Referința e adresa, nu obiectul. Mai multe referințe pot arăta către un singur obiect.
- La obiecte și `String` folosește `.equals()` pentru conținut, nu `==`.